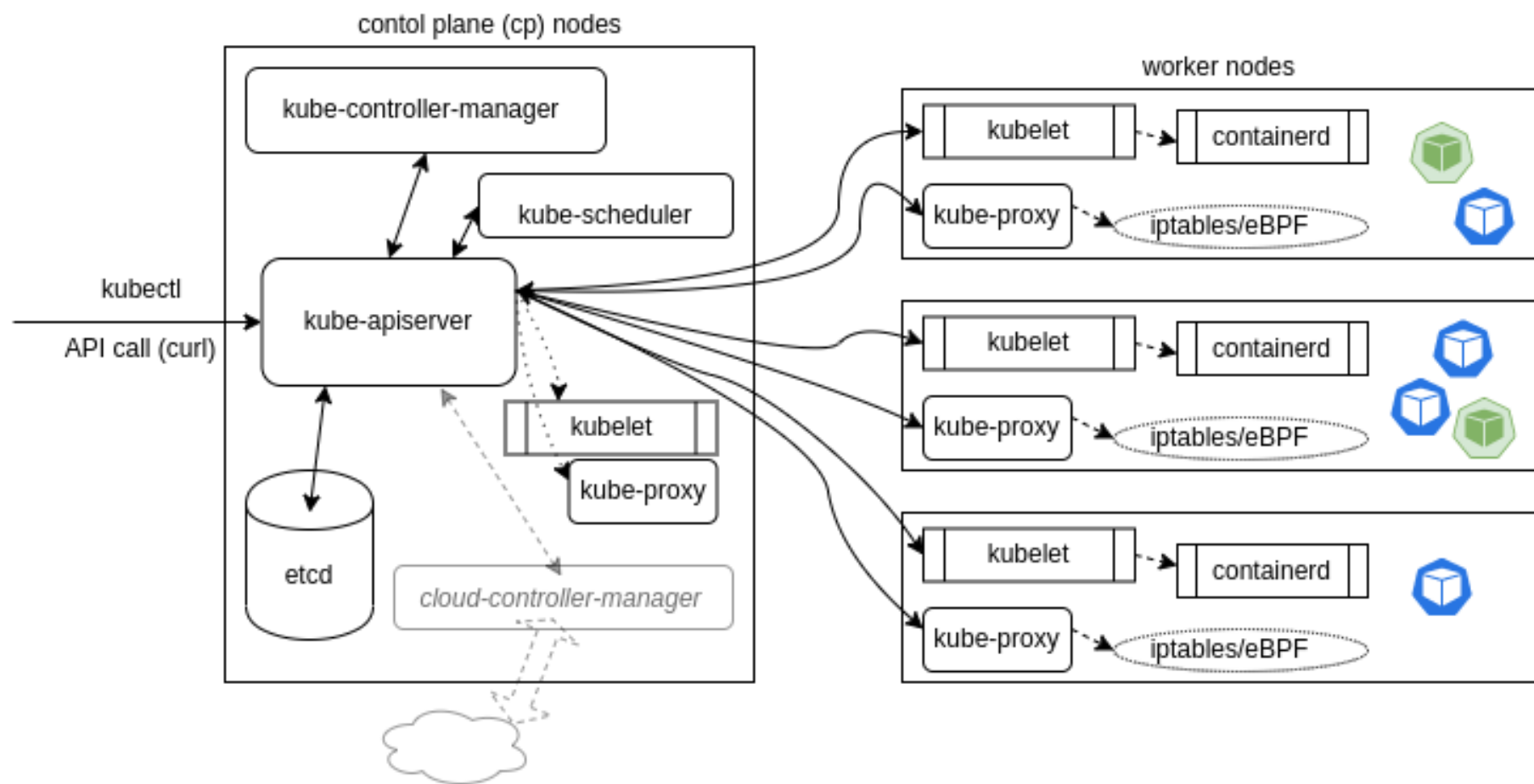
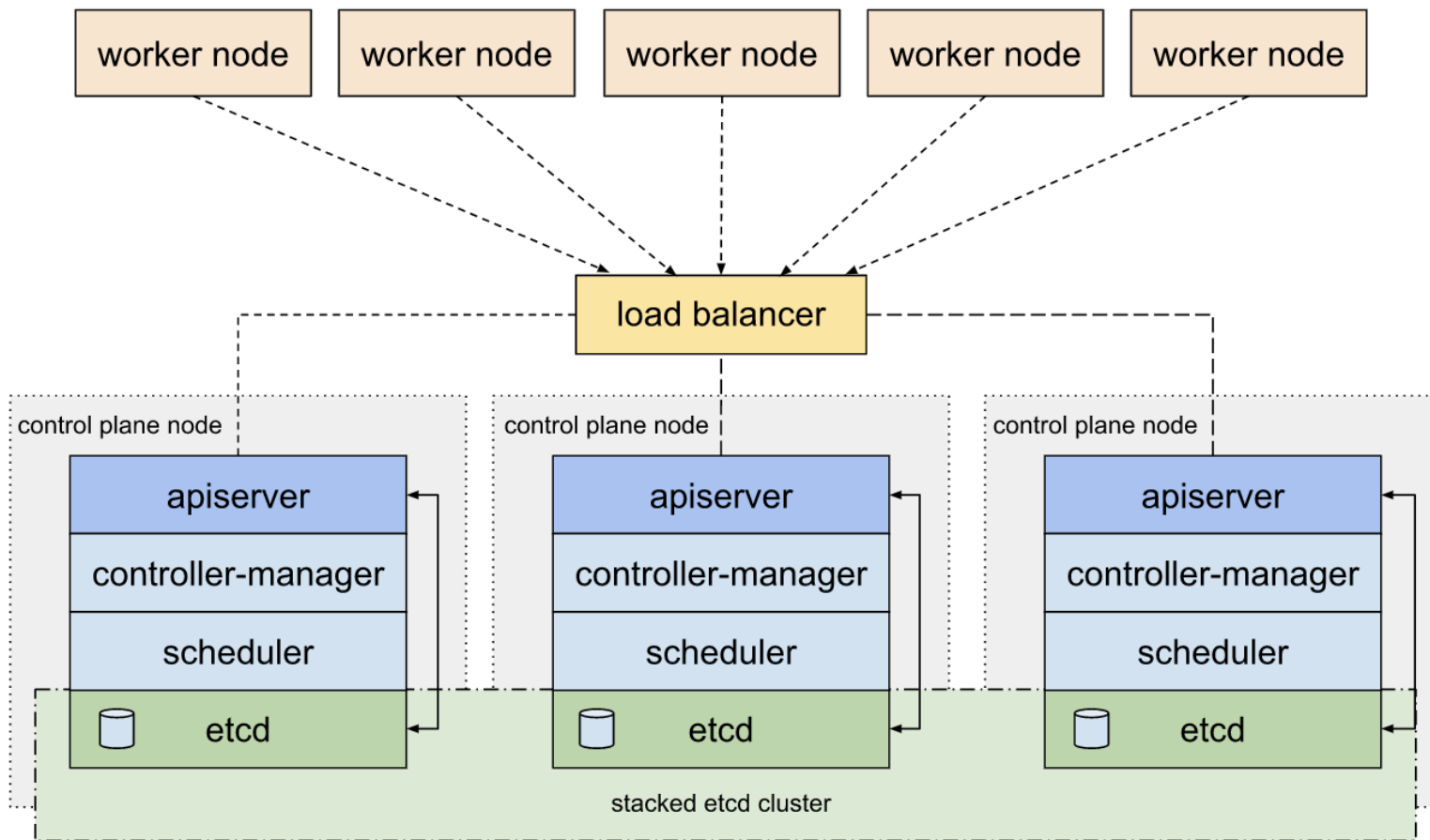
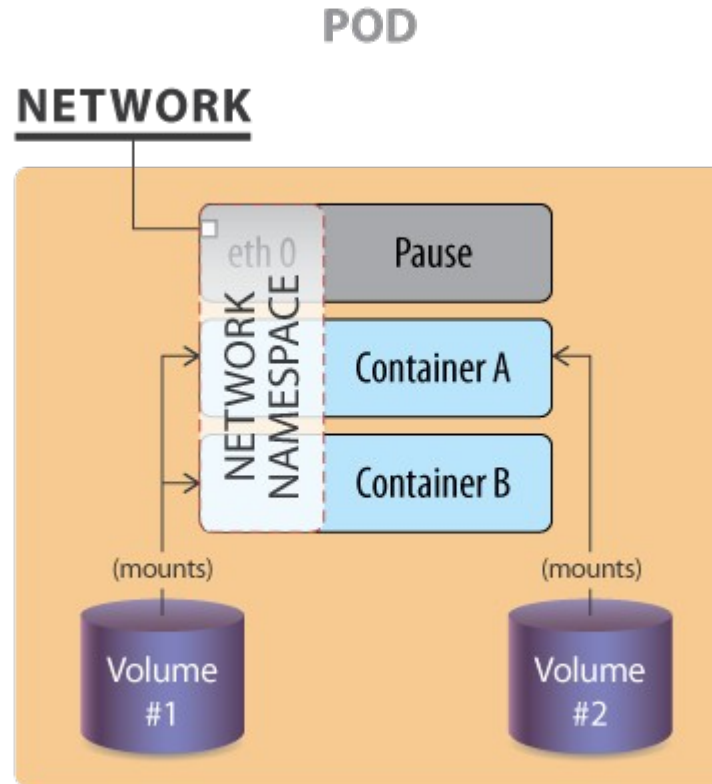
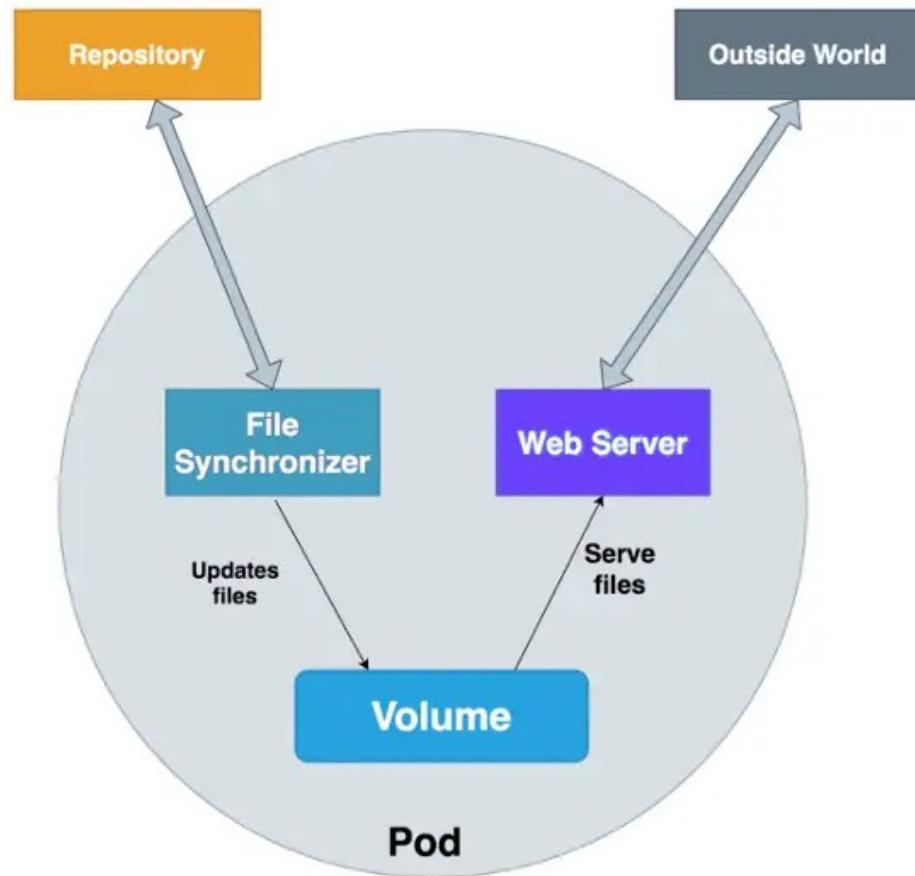


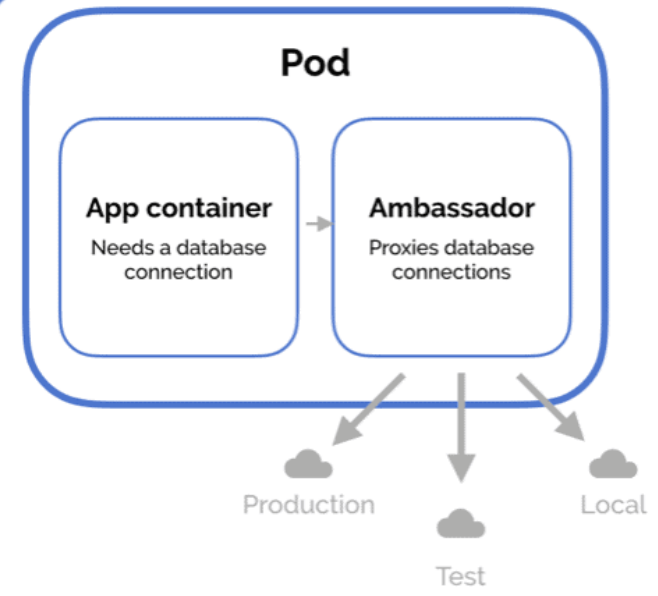
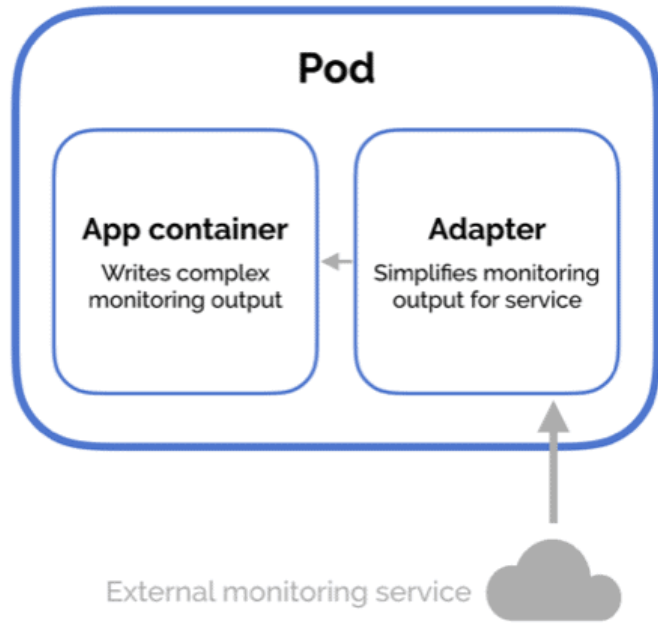
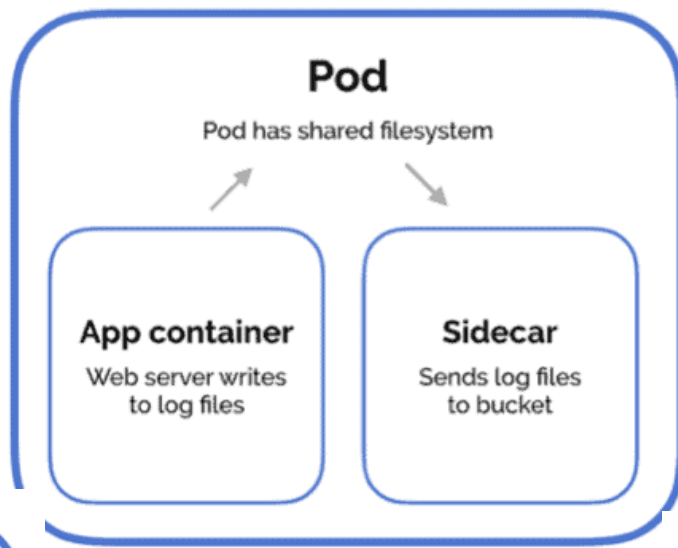




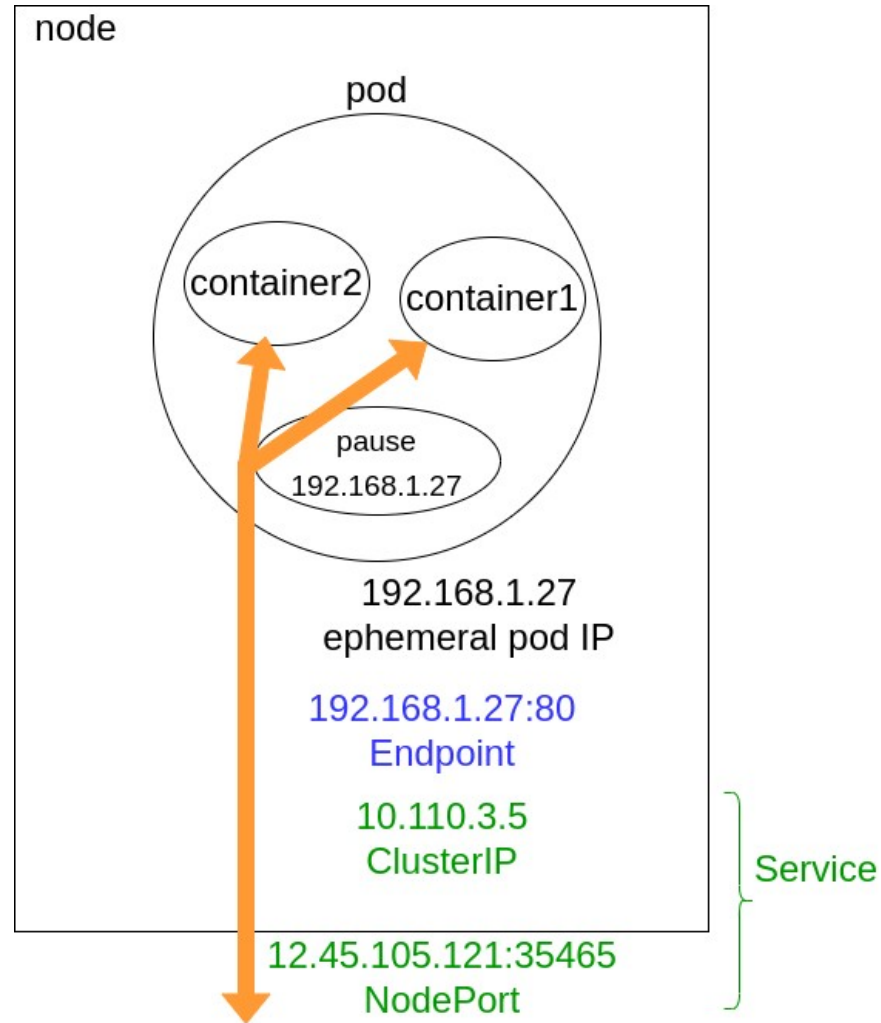
Single IP per Pod



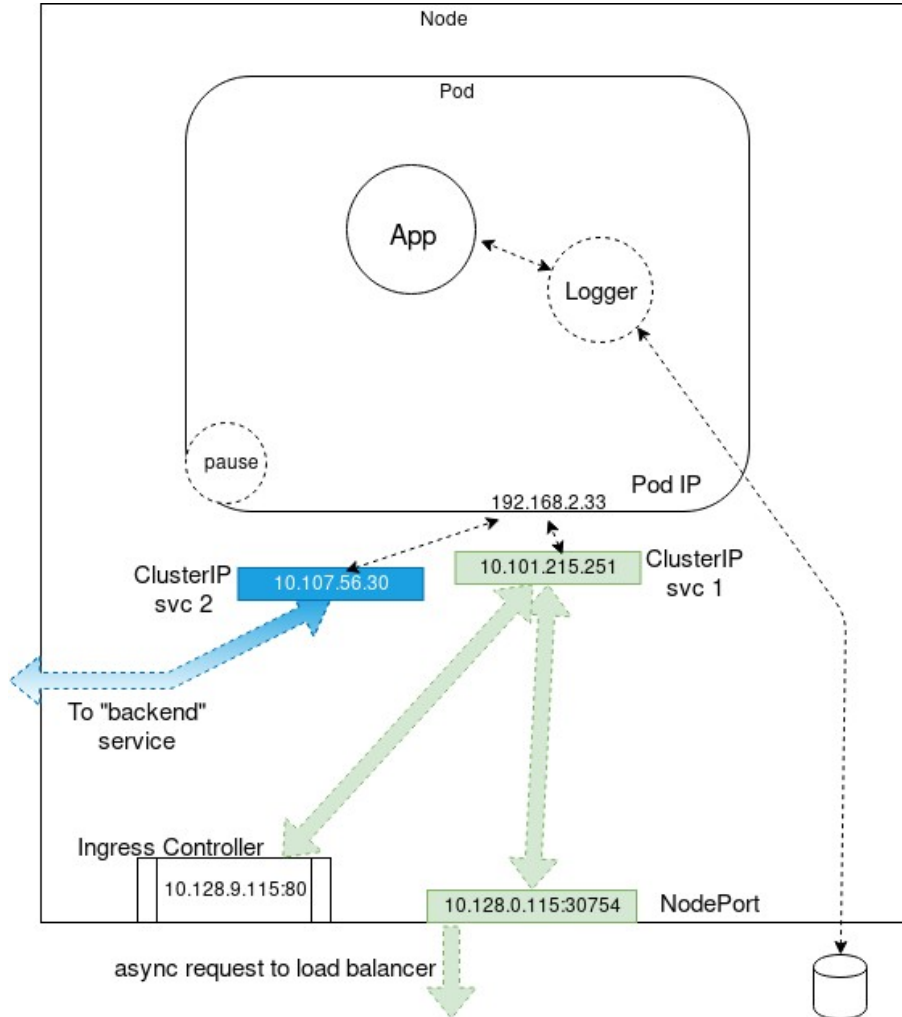




Container to Outside Path



Services



We can use a service to connect one pod to another, or to outside of the cluster.

Blue/Green Strategy

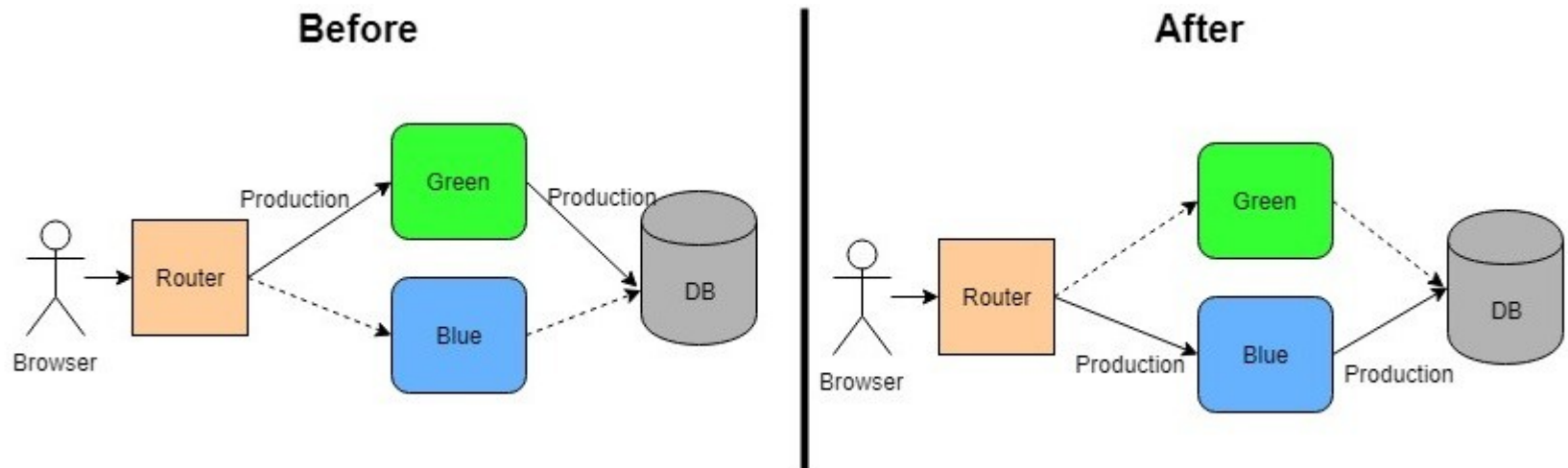


Blue-green deployment is a release **strategy** used in software deployment where you have two identical environments, often referred to as "blue" and "green."

Only one of these environments, known as the active environment, serves live traffic at any given time, while the other environment remains idle.



The basic idea behind blue-green deployments is to minimize downtime and risk by ensuring that there's always a stable environment available for users.





1. **Initial Setup**: Initially, both the blue and green environments are set up and configured identically. The blue environment serves as the active production environment, while the green environment is idle.
2. **Deployment**: When you need to deploy a new version of your application, you deploy it to the inactive environment (e.g., the green environment) without impacting the active environment (e.g., the blue environment). This allows you to thoroughly test the new version in isolation before making it live.
3. **Testing**: Once the new version is deployed to the green environment, you perform thorough testing to ensure that everything works as expected. This may include automated tests, manual QA, performance testing, etc.



4. **Switch Traffic**: Once the new version has been thoroughly tested and validated, you switch the traffic from the active environment (e.g., blue) to the newly deployed environment (e.g., green). This is typically done using some form of traffic routing or load balancer configuration.
5. **Validation**: After switching the traffic, you continue to monitor the newly active environment (e.g., green) to ensure that it's performing as expected and that there are no issues affecting users.
6. **Rollback (if necessary)**: In case any issues are detected after the switch, you can quickly revert the traffic back to the previous environment (e.g., blue) to minimize downtime and impact on users. This rollback process is usually straightforward since the previous environment remains intact and ready to serve traffic.

The key benefits of blue-green deployments include

a) reduced downtime

b) minimal risk,

c) ability to quickly roll back changes if issues arise.

This deployment strategy is commonly used in continuous delivery pipelines and can be implemented manually or automated using deployment tools and platforms like Kubernetes.

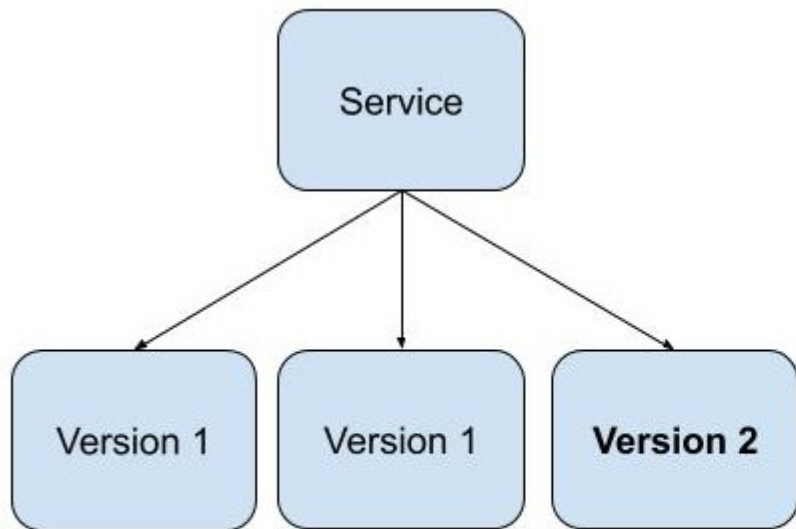
Canary deployments



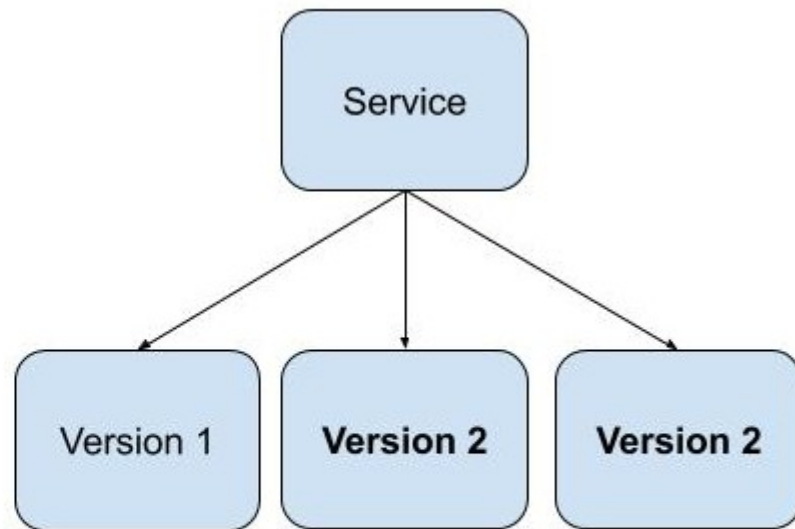
Canary deployment is a release strategy used in software deployment, particularly in the context of continuous delivery and continuous deployment pipelines.

The term *"canary" comes from the practice of using canaries in coal mines to detect dangerous gases.*

Similarly, in software deployment, a canary release involves deploying a new version of an application to a small subset of users or servers first to assess its performance and detect any issues before rolling it out to the wider audience.



Stage 1: 33% of traffic
handled by canary



Stage 2: 66% of traffic
handled by canary

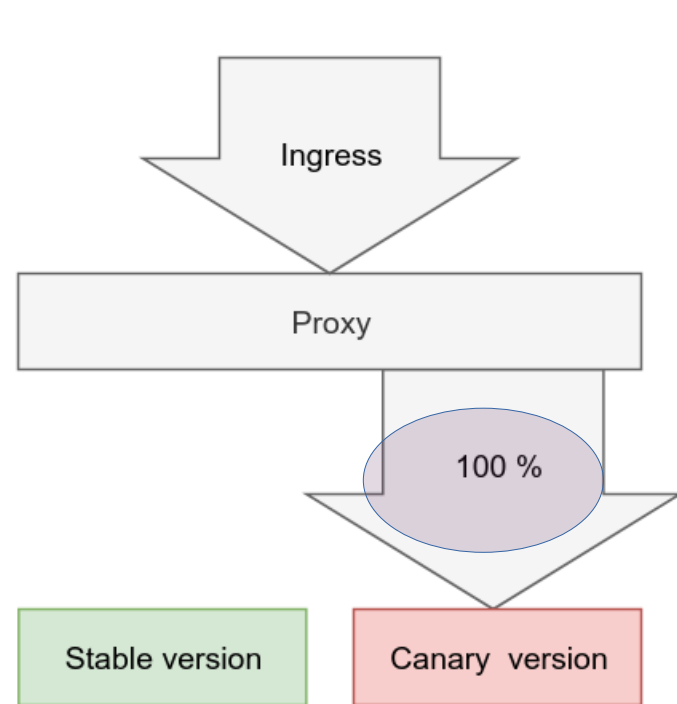
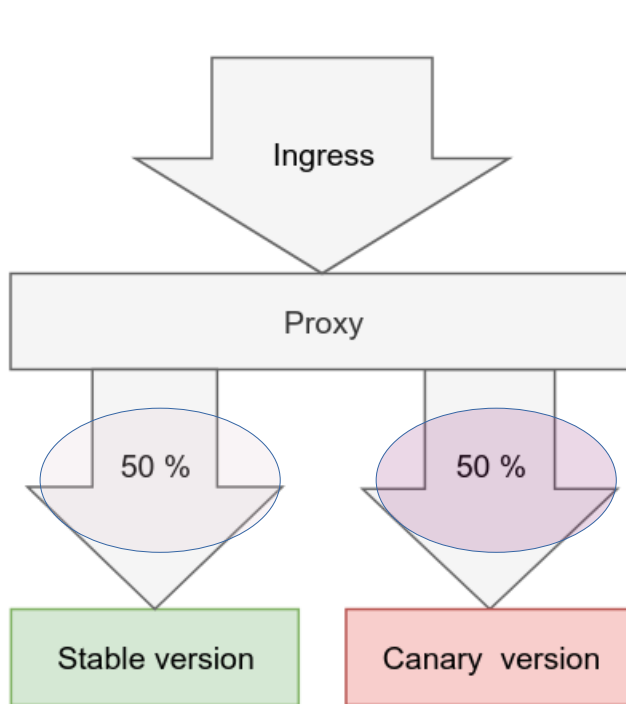
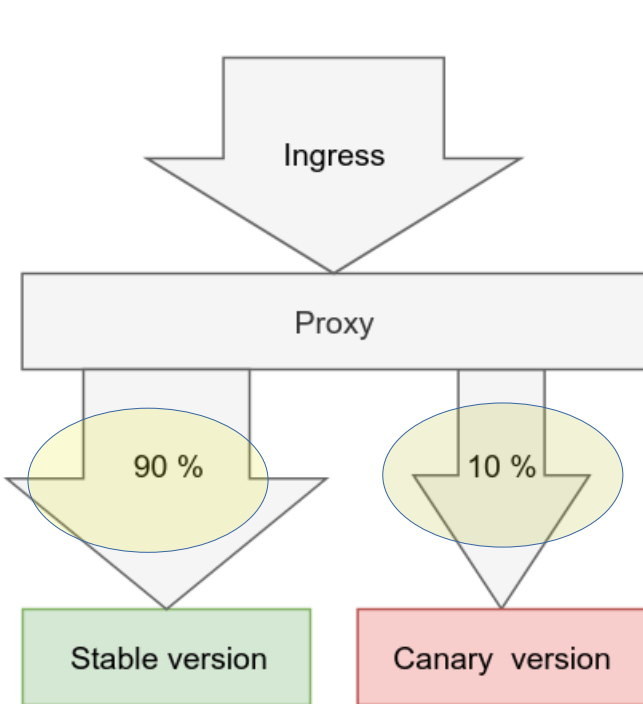
How canary deployments typically work:

1. **Initial Deployment**: Initially, the existing version of the application is deployed and serving production traffic.
2. **New Version Deployment**: When a new version of the application needs to be deployed, it's rolled out gradually to a small subset of users or servers. This subset is often referred to as the "canary group" or "canary deployment."
3. **Monitoring and Validation**: While the new version is deployed to the canary group, you closely monitor its performance, reliability, and other key metrics. This monitoring helps detect any issues or anomalies that may arise with the new version.

4. ****Gradual Rollout****: Assuming the new version performs well in the canary group, you gradually increase its rollout to a larger audience or more servers. This rollout can be automated based on predefined criteria or thresholds, such as error rates, latency, or user engagement metrics.

5. ****Validation and Verification****: Throughout the rollout process, you continue to monitor and validate the new version's performance and stability. If any issues are detected, you can quickly roll back the deployment to minimize impact on users.

6. ****Full Rollout or Rollback****: Once the new version has been thoroughly validated and verified, you can complete the rollout by deploying it to the entire user base or all servers. Alternatively, if issues are detected during the rollout, you can rollback to the previous version to maintain service reliability.

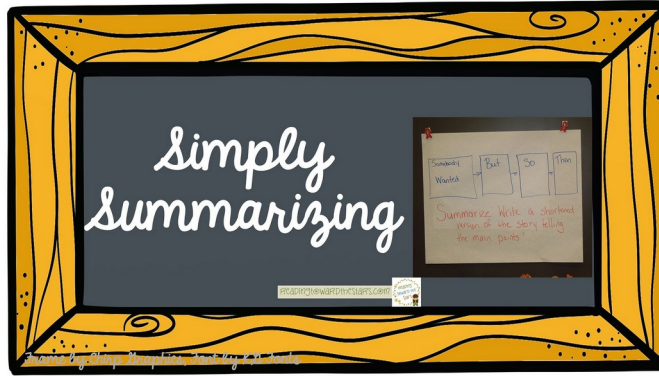




BENEFITS

key benefits of canary deployments include:

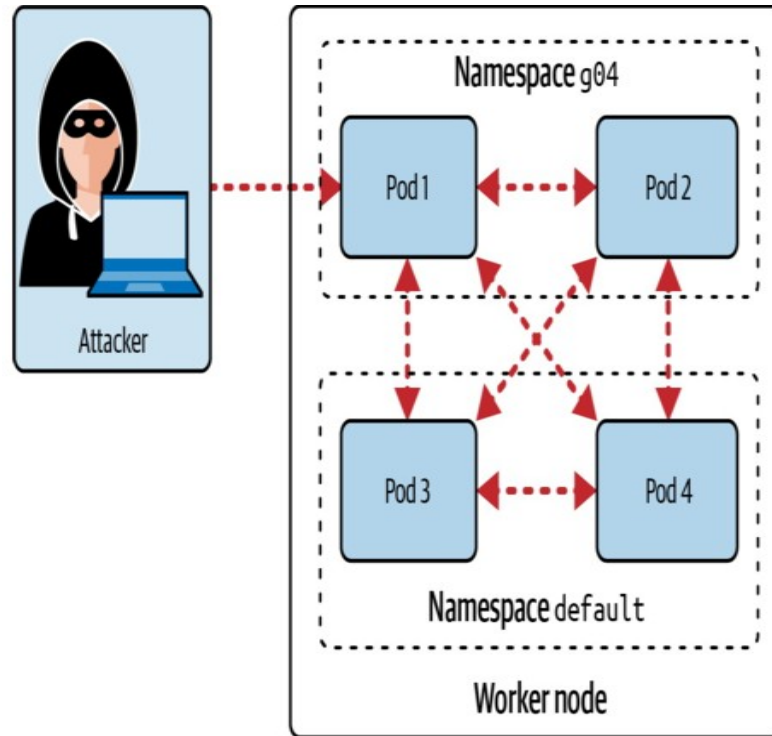
- **Risk Mitigation**: By gradually rolling out the new version to a small subset of users or servers first, you can detect and mitigate any issues before they affect the wider audience.
- **Faster Feedback**: Canary deployments provide faster feedback on the new version's performance and reliability, allowing you to make informed decisions about whether to proceed with the rollout or rollback.
- **Reduced Downtime**: By detecting and resolving issues early in the deployment process, you can minimize downtime and service disruptions for your users.



Canary deployments are commonly used in combination with automated testing, monitoring, and rollback mechanisms to ensure smooth and reliable software releases.

They are often implemented using deployment tools and platforms like Kubernetes, which provide features for traffic routing, version management, and automated rollout strategies.

CKS



NodePort vs LoadBalancer vs Ingress controller

Kubernetes offers multiple options for exposing services to external networks.

The main options are

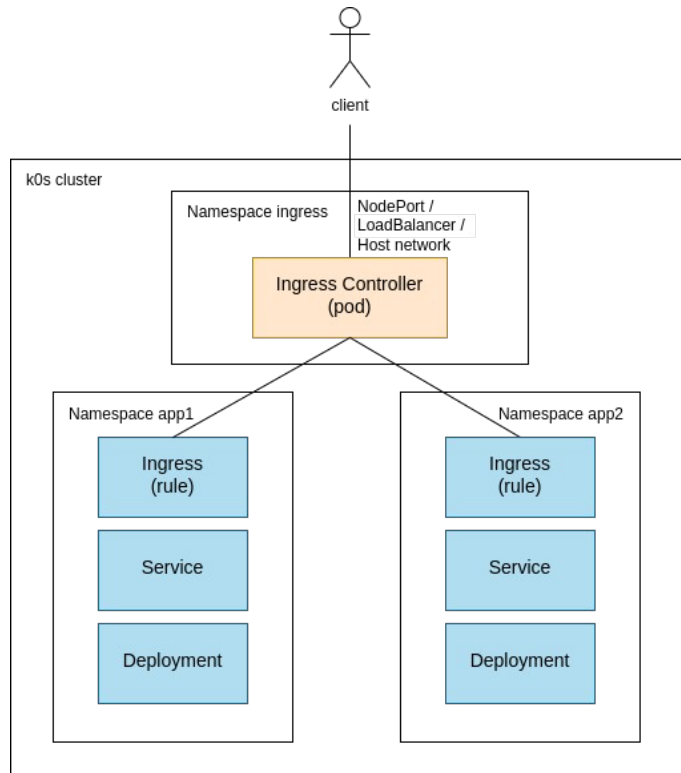
1) NodePort, 2) LoadBalancer 3) Ingress controller.

NodePort, as the name says, means that a port on a node is configured to route incoming requests to a certain service.

The port range is limited to 30000-32767, so you cannot expose commonly used ports like 80 or 443 with NodePort.

LoadBalancer is a service, which is typically implemented by the cloud provider as an external service (with additional cost). Load balancers can also be installed internally in the Kubernetes cluster with MetalLB, which is typically used for bare-metal deployments. Load balancer provides a single IP address to access your services, which can run on multiple nodes.

Ingress controller helps to consolidate routing rules of multiple applications into one entity. Ingress controller is exposed to an external network with the help of NodePort, LoadBalancer or host network. You can also use Ingress controller to terminate TLS for your domain in one place, instead of terminating TLS for each application separately.

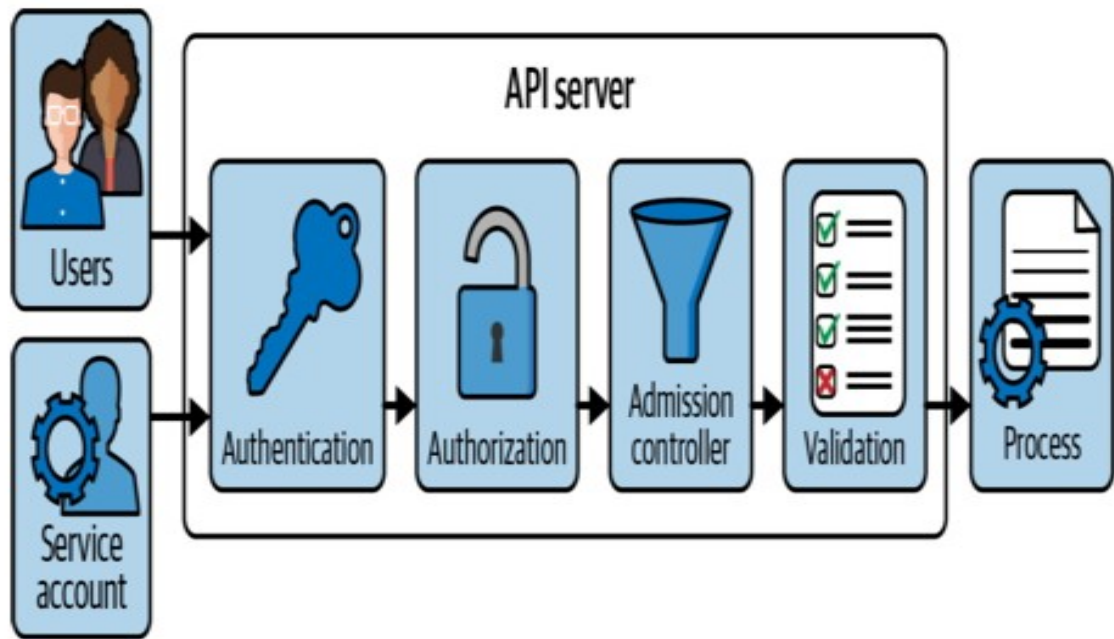


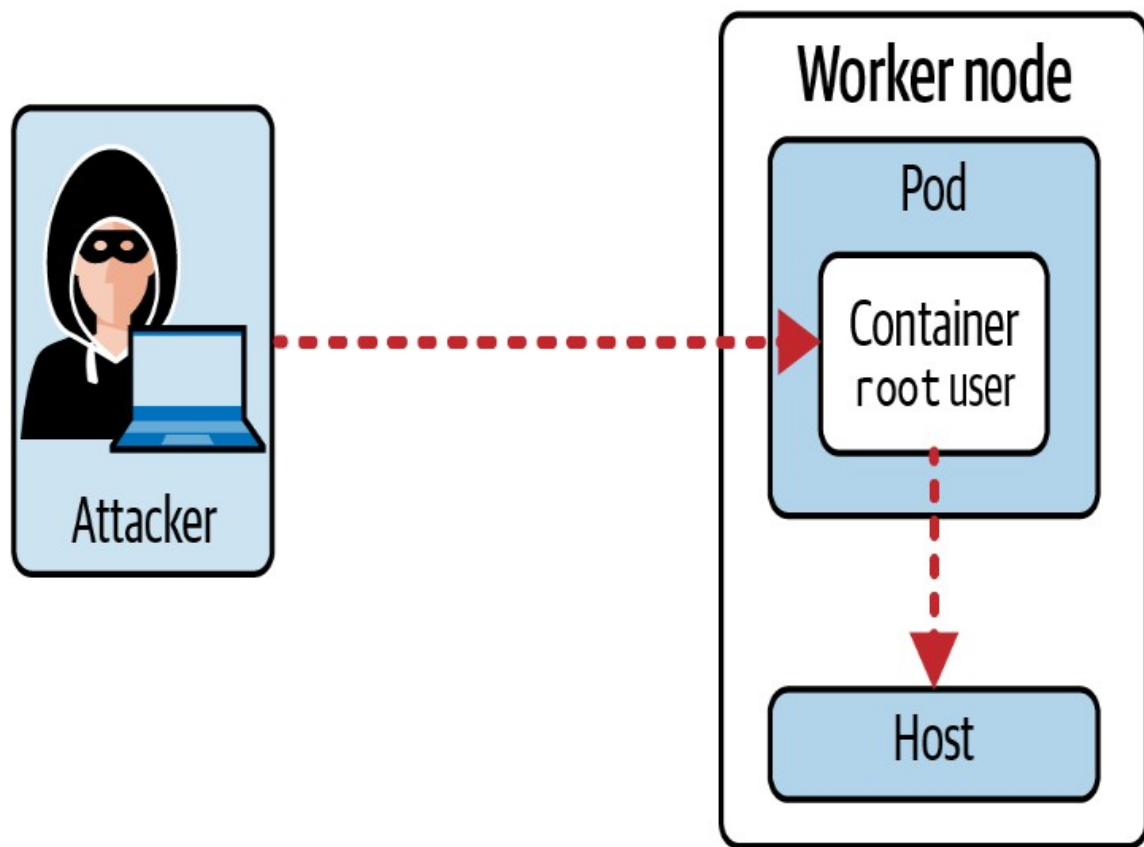
Restricting access to the Kubernetes API

Configuring role-based access control (RBAC) to minimize exposure

Exercising caution in using service accounts

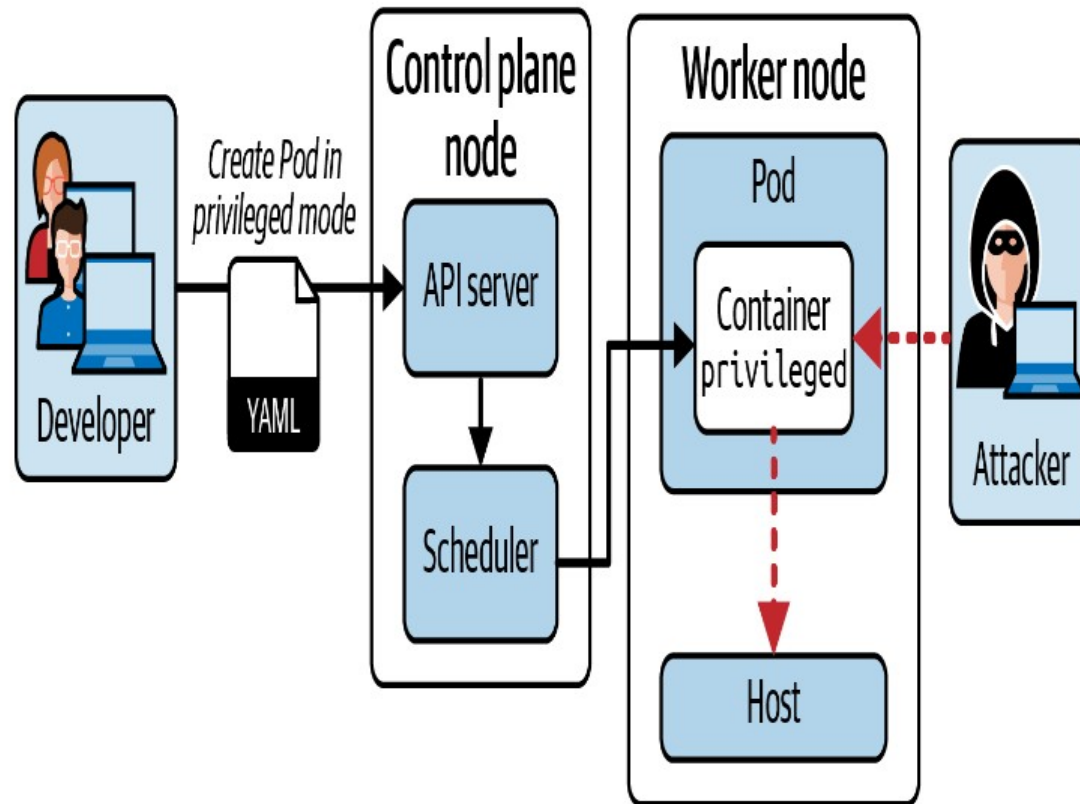
Updating Kubernetes frequently





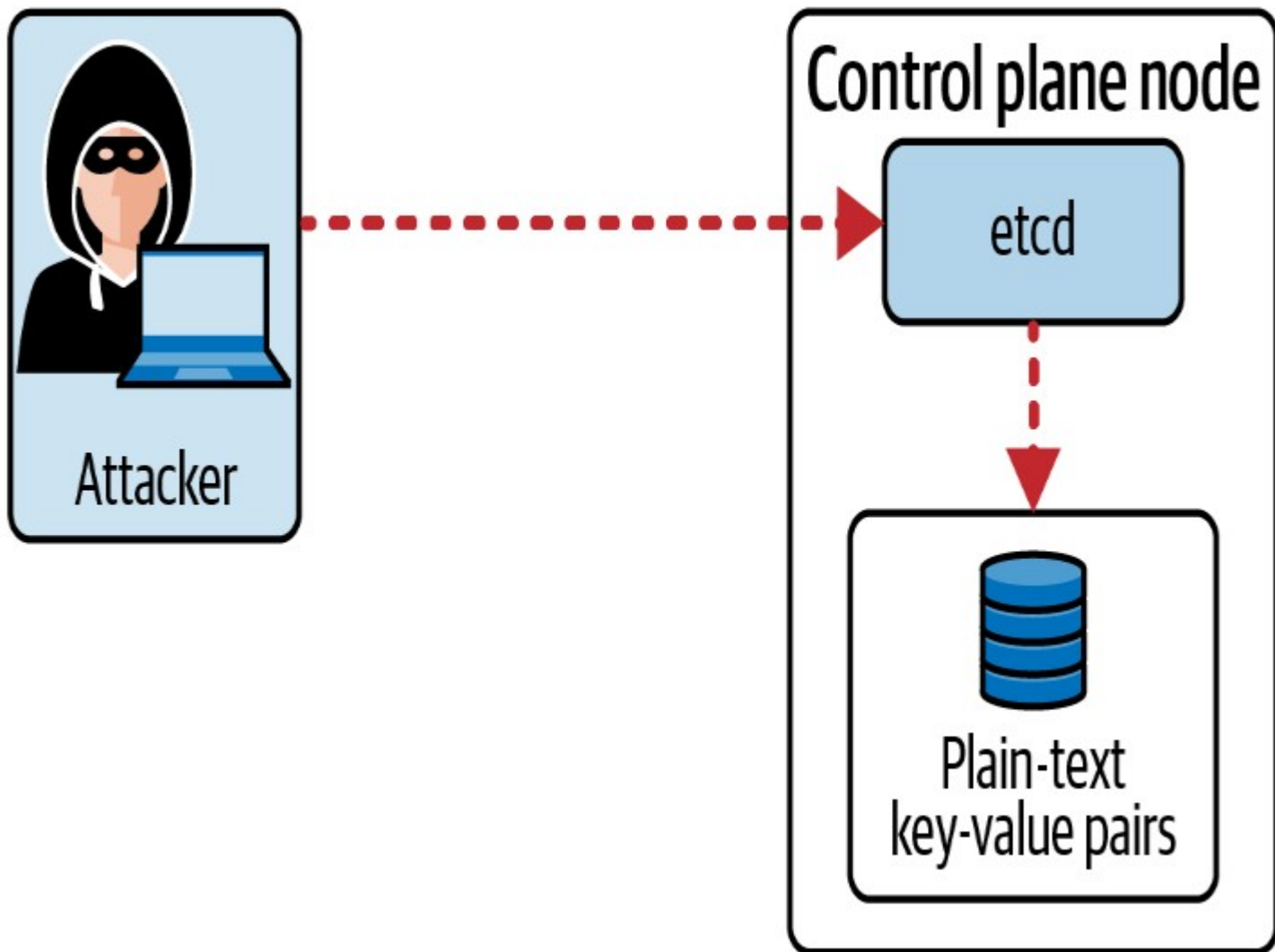
- The user ID that should be used to run the Pod and/or container
- The group ID that should be used for filesystem access
- Granting a running process inside the container some privileges of the root user but not all of them

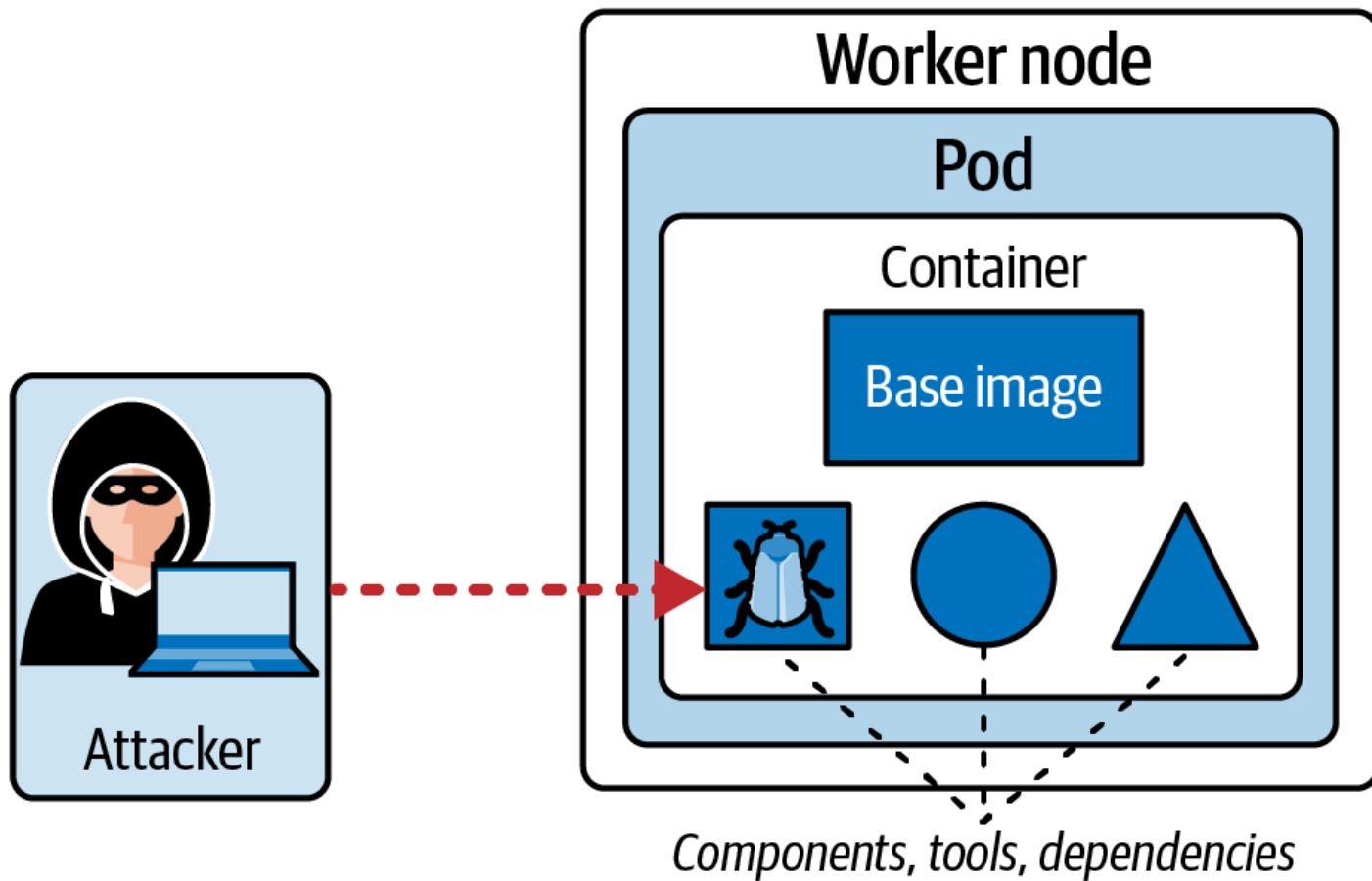
- Processes within a container almost have the same privileges as processes on the host.
- The container has access to all devices on the host.
- The root user in the container has similar privileges to the `root` user on the host.
- All directories on the host's filesystem can be mounted in the container.
- Kernel settings can be changed, e.g., by using the `sysctl` command.

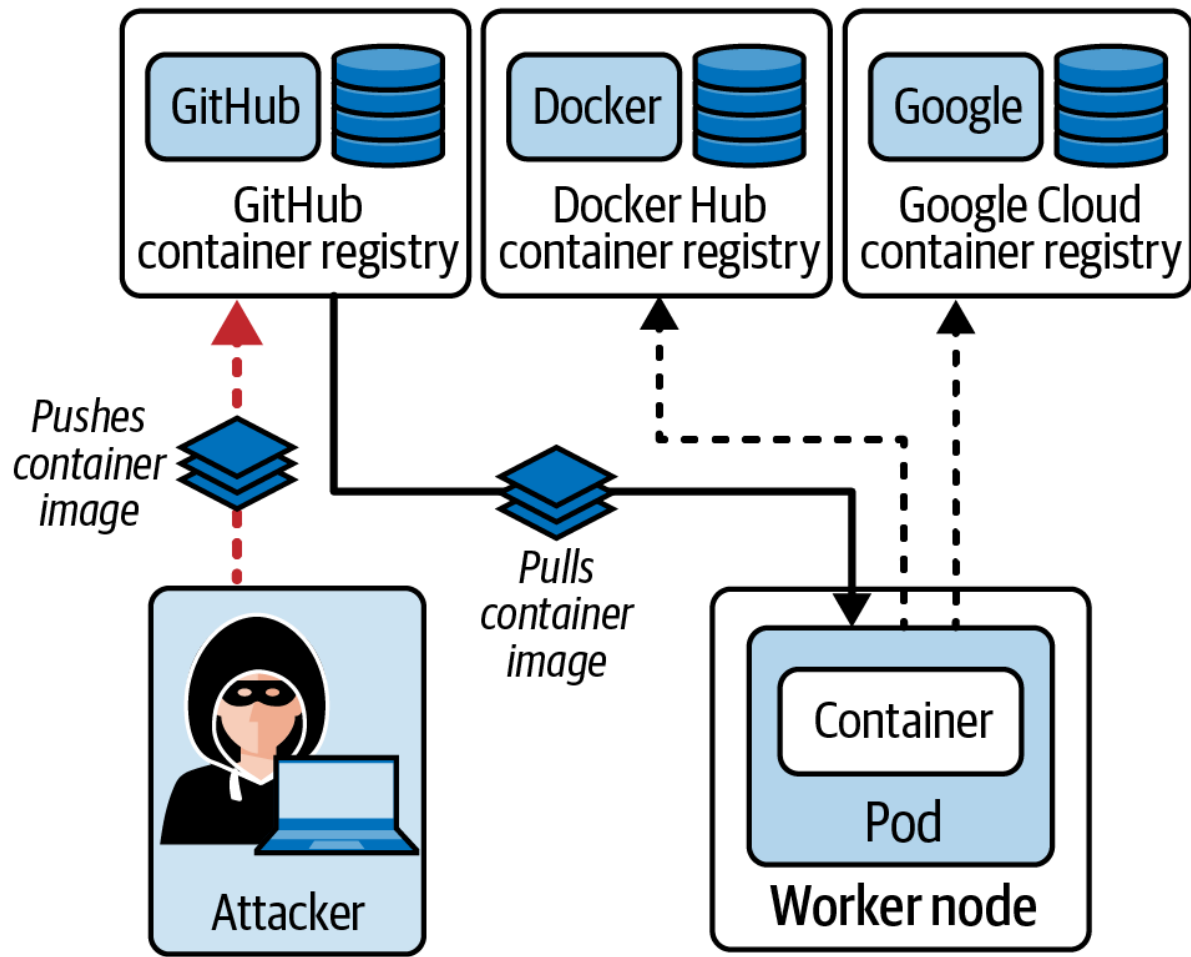


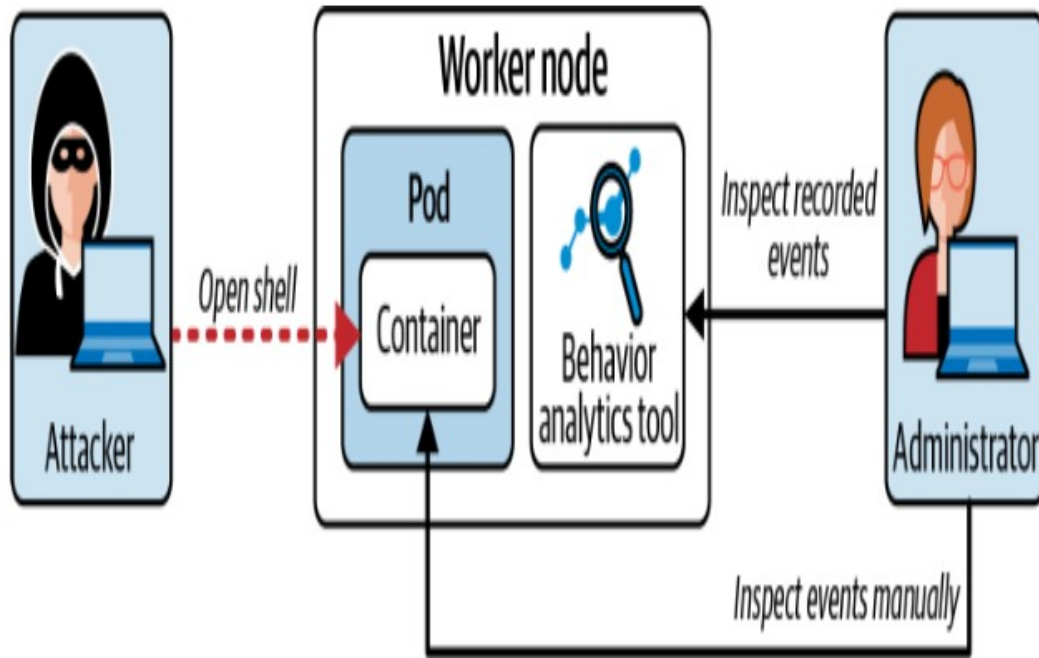
Mode	Behavior
enforce	Violations will cause the Pod to be rejected.
audit	Pod creation will be allowed. Violations will be appended to the audit log.
warn	Pod creation will be allowed. Violations will be rendered on the console.

Level	Behavior
privileged	Fully unrestricted policy.
baseline	Minimally restrictive policy that covers crucial standards.
restricted	Heavily restricted policy following best practices for hardening Pods from a security perspective.









Performing behavior analytics to detect malicious activities

Performing deep analytical investigation and identification

Ensuring immutability of containers at runtime

Using audit logs to monitor access

Reading or writing files at specific locations in the filesystem

Opening a shell binary for a container, such as /bin/bash to open a bash shell

An attempt to make a network call to undesired URLs

